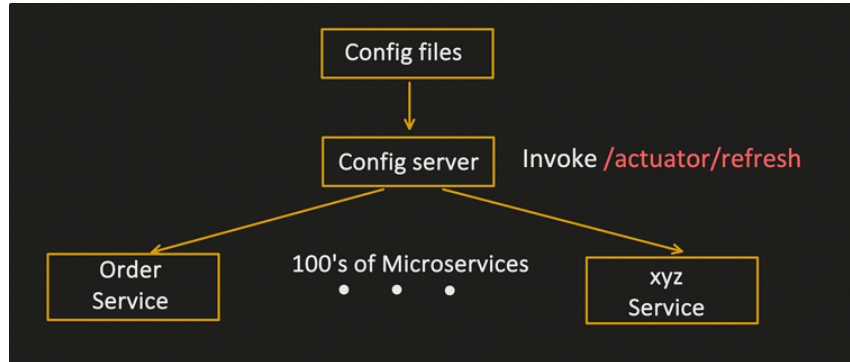


Spring Cloud Bus

This is where we end in the previous topic of @RefreshScope:



Now do I have to Invoke `/actuator/refresh` for these 100's of Microservices

If we are using Actuator approach to create a refresh event then YES.

But, if we want , only if I do REFRESH at ConfigServer and all the microservices get notified and they should also refresh themselves.

Then its also possible with “**Spring Cloud Bus**”.

Let's start:

First, see how event is published **within 1 application** itself:

```
1 package com.concepts.WithoutSpringCloudBus;
2
3 import org.springframework.context.ApplicationEvent;
4
5 public class MyCustomEvent extends ApplicationEvent {
6     private final String message;
7
8     public MyCustomEvent(Object source, String message) {
9         super(source);
10        this.message = message;
11    }
12
13    public String getMessage() {
14        return message;
15    }
16 }
```

- “MyCustomEvent” event which we need to publish to listeners.
- This event class should extends ApplicationEvent, as “publishEvent()” method accepts ApplicationEvent type only.

```
1 package com.concepts.WithoutSpringCloudBus;
2
3 import org.springframework.beans.factory.annotation.Autowired;
4 import org.springframework.context.ApplicationEventPublisher;
5 import org.springframework.stereotype.Service;
6
7 @Service
8 public class MyEventPublisher {
9
10    @Autowired
```

Concept && Coding

```

11     private ApplicationEventPublisher publisher;
12
13     public void publish(String msg) {
14         publisher.publishEvent(new MyCustomEvent(this, msg));
15     }
16 }

```

- *ApplicationEventPublisher* interface abstract the event publishing functionality.
- We can invoke method “publishEvent(ApplicationEvent object)” and it will take care of publishing the event to all the listeners.

```

1  package com.concepts.WithoutSpringCloudBus;
2
3  import org.springframework.context.event.EventListener;
4  import org.springframework.stereotype.Component;
5
6  @Component
7  public class MyEventListener {
8
9      @EventListener
10     public void handleEvent(MyCustomEvent event) {
11         System.out.println("Received event message: " +
12             event.getMessage());
13     }
14 }

```

- *MyEventListener*, listening to *MyCustomEvent* event.
- During application startup, framework maintains the list

something like :

MyCustomEvent → *MyEventListener*, *Listener2*,

SomeOtherEvent → *Listener3*, *Listener4*,

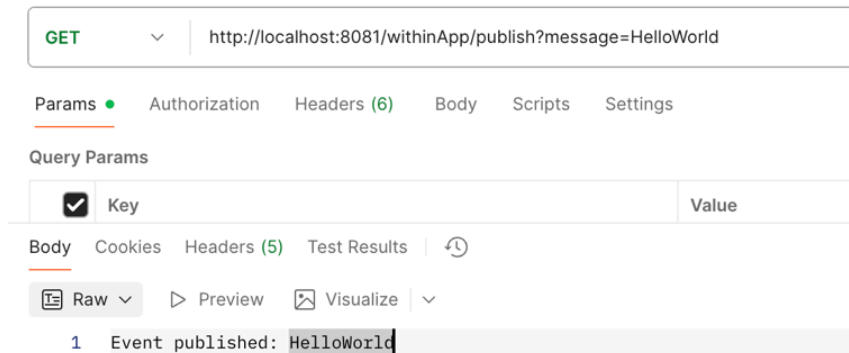
Concept && Coding

That's it, lets add the controller and test if event publishing works within 1 application or not:

```

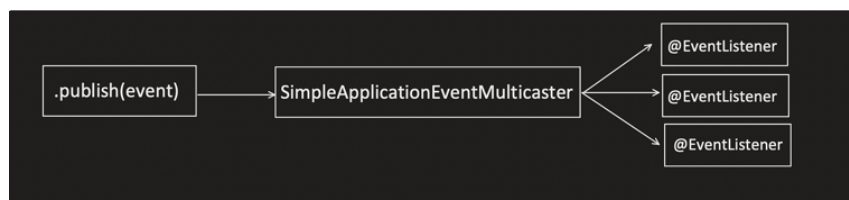
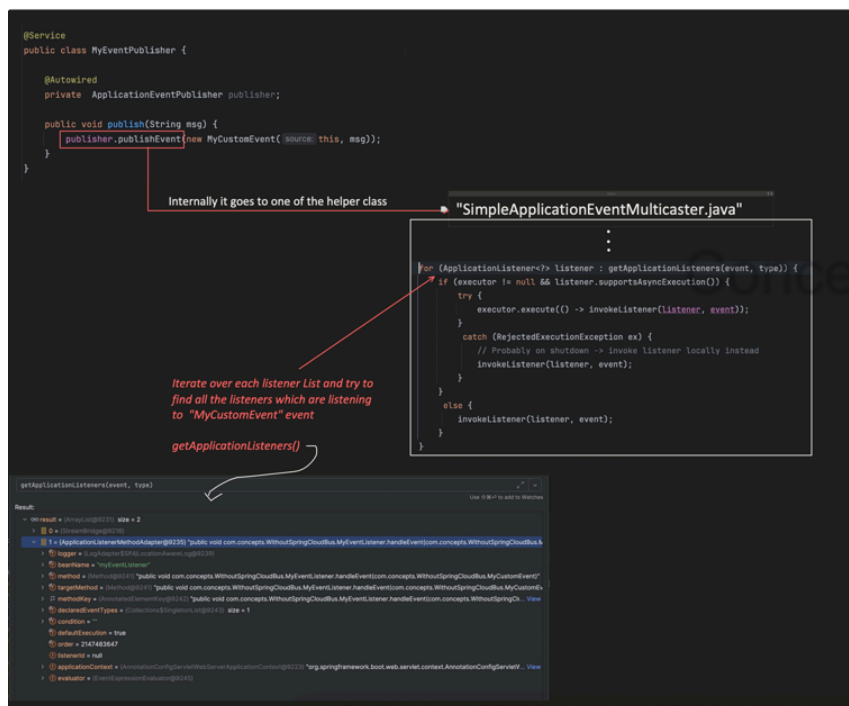
1  package com.concepts.WithoutSpringCloudBus;
2
3  import org.springframework.beans.factory.annotation.Autowired;
4  import org.springframework.web.bind.annotation.GetMapping;
5  import org.springframework.web.bind.annotation.RequestParam;
6  import org.springframework.web.bind.annotation.RestController;
7
8  @RestController
9  public class EventController {
10
11      @Autowired
12      private MyEventPublisher publisher;
13
14      @GetMapping("/withinApp/publish")
15      public String publish(@RequestParam String message) {
16          publisher.publish(message);
17          return "Event published: " + message;
18      }
19 }

```



```
2025-11-11T12:22:36.207+05:30 INFO 68342 --- [producer-service] [nio-8081-exec-1] o.s.web.servlet.DispatcherServlet
2025-11-11T12:22:36.208+05:30 INFO 68342 --- [producer-service] [nio-8081-exec-1] o.s.web.servlet.DispatcherServlet
Received event message: HelloWorld
```

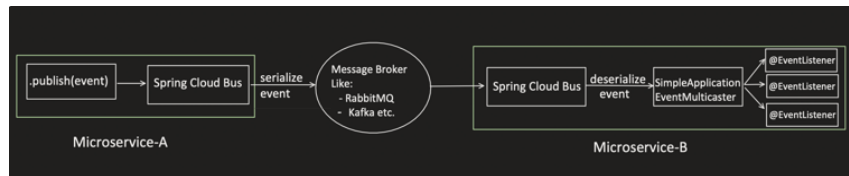
How its working?



So very similar to Observer design pattern (in LLD) and its all Sync manner and within an application.

But how to handle, when Publisher and Listener is not in a same app and they are in different microservices?

Here comes: **Spring Cloud Bus**



Spring Cloud Bus, abstract all the details regarding message broker:

If we are using RabbitMQ (AMQP protocol):

- we don't have to create "exchange" and "queue".
- No logic to bind the "queue" and "exchange" with routing key etc..

If we are using Kafka:

- We don't have to create topics
- We don't have to create partitions etc..

Everything is created and managed by Spring cloud bus and its all abstracted to us.

Spring Cloud Bus is just as wrapper above the Message Broker like Kafka and RabbitMQ. And just pass the event to the broker and leave up to broker and its configuration for retain messages or retry or what to do in case of failure etc.

Designed for:

- Low volume and non critical messages like refresh config, clear cache etc..
- Not recommended for very critical and high volume system.

If we need full control like over Topic/Queues creation, Retry mechanism, Message persistence logic, ACK etc. then its recommended to use "Spring Cloud Stream" or Kafka Template etc..

Not recommended to use Spring cloud bus.

Now lets set up: Custom Event using Spring Cloud Bus

Step1:

Since Spring cloud bus is using RabbitMQ or Kafka in background, we need to set up anyone for testing:

Installation of RabbitMQ:

- brew update
- brew install rabbitmq

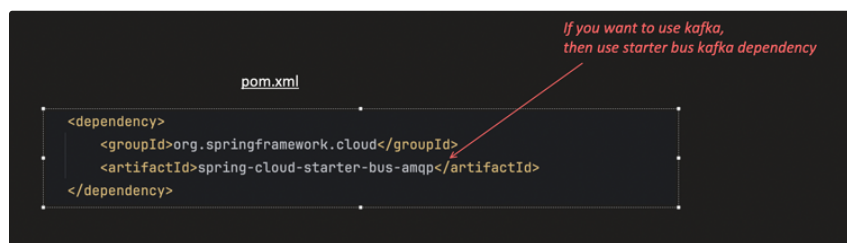
Start up of RabbitMQ:

- brew services start rabbitmq

RabbitMQ Started:



Step2: Publisher



```

1 package com.concepts.WithSpringCloudBus;
2
3 import org.springframework.cloud.bus.event.RemoteApplicationEvent;
4
5 public class MyRemoteCustomEvent extends RemoteApplicationEvent {
6
7     public MyRemoteCustomEvent() {} // default constructor for
        deserialization
8
9     public MyRemoteCustomEvent(Object source, String originService,
        String destination, String message) {
10         super(source, originService, destination);
11         this.message = message;
12     }
13
14     private String message;
15
16     public String getMessage() {
17         return message;
18     }
19 }

```

- **MyRemoteCustomEvent** which we need to publish to listeners.
- This event class should extends **RemoteApplicationEvent**, then only it will be intercepted by Spring Cloud Bus.

```

public MyRemoteCustomEvent(Object source, String originService, String destination, String
message) {

```

```

...
}

```

- **source**: details about this event class and object.
- **originService**: which application is publishing the event.
- **Destination**: determines the target Microservice(s).

*: broadcast to all or Specific Application name.

Very Important:

Spring cloud bus framework while publishing checks, I should only publish the event which is generated by me, I

should not publish on behalf of someone else. This is required to prevent infinite looping.

This **originService** name tells, this msg is generated by which service.

originService name should match with this service Bus ID, else publish will not happen.

```
1 package com.concepts.WithSpringCouldBus;
2
3 import org.springframework.beans.factory.annotation.Autowired;
4 import org.springframework.beans.factory.annotation.Value;
5 import org.springframework.context.ApplicationEventPublisher;
6 import org.springframework.stereotype.Service;
7
8 @Service
9 public class MyRemoteEventPublisher {
10
11     @Autowired
12     private ApplicationEventPublisher publisher;
13
14     @Value("${spring.cloud.bus.id}")
15     String myBusID;
16
17     public void publish(String msg) {
18         publisher.publishEvent(new MyRemoteCustomEvent(this, myBusID,
19             "*", msg));
20     }
21 }
```

@Value("\${[spring.cloud.bus.id](#)}")

String myBusID;

Reading the bus id from properties.

If we don't want this, we can also use like: "producer-service:****"

Then internally framework will only compare the application name.

Port and UUID matching is skipped.

publisher.publishEvent(new MyRemoteCustomEvent(this, myBusID, "*", msg));

All info regarding message broker (either kafka or RabbitMQ)is abstracted. We just publish.

Controller class, just to invoke the flow:

```
1 package com.concepts.WithSpringCouldBus;
2
3 import org.springframework.beans.factory.annotation.Autowired;
4 import org.springframework.web.bind.annotation.GetMapping;
5 import org.springframework.web.bind.annotation.RequestParam;
6 import org.springframework.web.bind.annotation.RestController;
7
8 @RestController
9 public class RemoteEventController {
10
11     @Autowired
12     MyRemoteEventPublisher publisher;
13
14     @GetMapping("/publish")
15     public String publish(@RequestParam String message) {
16         publisher.publish(message);
17         return "Event published: " + message;
18     }
19 }
```

Concept && Coding

Step3: Listener

pom.xml

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-bus-amqp</artifactId>
</dependency>
```

application.properties

```
spring.application.name=consumer-service
server.port=8082

spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest
```

Listener:

```
@Component
public class MyRemoteCustomEventListener {

    @EventListener
    public void handle(MyRemoteCustomEvent event) {
        System.out.println("Received remote event in client-service: " + event.getMessage());
    }
}
```

This Remote Event class, we can put in common place, so that both Publisher and Listener can access it

Or

Duplicate at both the place, for now I have duplicated.

```
@SpringBootApplication
@RemoteApplicationEventScan(basePackages = "com.concepts")
public class CloudBusConsumerApp {
    public static void main(String[] args) {
        SpringApplication.run(CloudBusConsumerApp.class, args);
    }
}
```

*When Spring cloud bus, tries to do the deserialization (JSON → java object), it need to look for Object, so with **RemoteApplicationEventScan** we are just telling that look in this base package or its sub package.*

Lets start Publisher and Listener:

```

main) o.s.c.c.[tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
main) w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: Initialization completed in 448 ms
main) o.f.c.c.BeanFactoryMethodResolutionRegistry : Failed to locate function 'busConsumer' for function definition 'busConsumer'. Returning null.
main) o.s.c.s.w.DirectWritableAttributesChannel : Channel 'consumer-service.springCloudBusInput' has 1 subscriber(s).
main) o.s.i.endpoint.EventDrivenConsumer : Adding {logging-channel-adapter.org.springframework.integration.errorLogger} as a subscriber to the 'errorChannel' channel
main) o.s.i.channel.PublishSubscribeChannel : Channel 'consumer-service.errorChannel' has 1 subscriber(s).
main) o.s.i.endpoint.EventDrivenConsumer : started bean 'org.springframework.integration.errorLogger'
main) o.s.c.s.binder.DefaultBinderFactory : Creating binder: rabbit
main) o.s.c.s.binder.DefaultBinderFactory : Constructing binder child context for rabbit
main) o.s.c.s.binder.DefaultBinderFactory : Caching the binder: rabbit
main) o.s.s.r.p.RabbitExchangeQueueProvisioner : declaring queue for inbound: springCloudBus.anonymous.bM9P3eZu8080MeJlQgw, bound to: springCloudBus
main) o.s.s.r.p.cachingConnectionFactory : Attempting to connect to: [localhost:5672]
main) o.s.s.r.p.cachingConnectionFactory : Created new connection: rabbitConnectionFactory#53ea380b:0/SingleConnection@136f0e2 [delegate=amp://guest@127.0.0.1:5672/, LocalPort=56772]
main) o.s.c.s.stream.binder.BinderErrorChannel : Channel 'rabbit-1713223230.springCloudBusInput.errors' has 1 subscriber(s).
main) o.s.c.s.stream.binder.BinderErrorChannel : Channel 'rabbit-1713223230.springCloudBusInput.errors' has 2 subscriber(s).
main) o.s.i.s.i.AmqpInboundChannelAdapter : started bean 'inbound.springCloudBus.anonymous.bM9P3eZu8080MeJlQgw'
main) o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8082 (http) with context path '/'
main) com.concepts.CloudBusConsumerApp : Started CloudBusConsumerApp in 1.16 seconds (process running for 1.37s)

```

```

n) o.s.c.c.c.[tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
n) w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: Initialization completed in 496 ms
n) o.f.c.c.BeanFactoryMethodResolutionRegistry : Failed to locate function 'busConsumer' for function definition 'busConsumer'. Returning null.
n) o.s.c.s.w.DirectWritableAttributesChannel : Channel 'producer-service.springCloudBusInput' has 1 subscriber(s).
n) o.s.i.endpoint.EventDrivenConsumer : Adding {logging-channel-adapter.org.springframework.integration.errorLogger} as a subscriber to the 'errorChannel' channel
n) o.s.i.channel.PublishSubscribeChannel : Channel 'producer-service.errorChannel' has 1 subscriber(s).
n) o.s.i.endpoint.EventDrivenConsumer : started bean 'org.springframework.integration.errorLogger'
n) o.s.c.s.binder.DefaultBinderFactory : Creating binder: rabbit
n) o.s.c.s.binder.DefaultBinderFactory : Constructing binder child context for rabbit
n) o.s.c.s.binder.DefaultBinderFactory : Caching the binder: rabbit
n) o.s.b.r.p.RabbitExchangeQueueProvisioner : declaring queue for inbound: springCloudBus.anonymous.GJJf8A_25T0y2vtfWlW, bound to: springCloudBus
n) o.s.s.r.p.cachingConnectionFactory : Attempting to connect to: [localhost:5672]
n) o.s.s.r.p.cachingConnectionFactory : Created new connection: rabbitConnectionFactory#1cde37f4:0/SingleConnection@04defdc [delegate=amp://guest@127.0.0.1:5672/, LocalPort=56921]
n) o.s.c.s.stream.binder.BinderErrorChannel : Channel 'rabbit-1383667524.springCloudBusInput.errors' has 1 subscriber(s).
n) o.s.c.s.stream.binder.BinderErrorChannel : Channel 'rabbit-1383667524.springCloudBusInput.errors' has 2 subscriber(s).
n) o.s.i.s.i.AmqpInboundChannelAdapter : started bean 'inbound.springCloudBus.anonymous.GJJf8A_25T0y2vtfWlW'
n) o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8081 (http) with context path '/'
n) com.concepts.CloudBusProducerApp : Started CloudBusProducerApp in 1.231 seconds (process running for 1.445)

```

GET

http://localhost:8081/publish?message=HelloWorld

Params

Authorization

Headers (6)

Body

Scripts

Settings

Query Params

Key	Value
1	Event published: HelloWorld

Raw

Preview

Visualize

```

2025-11-11T14:32:26.692+05:30 INFO 91439 --- [consumer-service] [main] o.s.c.s.stream.binder.BinderErrorChannel : Channel 'rabbit-1713223230.springCloudBusInput.errors' has 2 subscriber(s).
2025-11-11T14:32:26.694+05:30 INFO 91439 --- [consumer-service] [main] o.s.i.s.i.AmqpInboundChannelAdapter : started bean 'inbound.springCloudBus.anonymous.bM9P3eZu8080MeJlQgw'
2025-11-11T14:32:26.706+05:30 INFO 91439 --- [consumer-service] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8082 (http) with context path '/'
2025-11-11T14:32:26.718+05:30 INFO 91439 --- [consumer-service] [main] com.concepts.CloudBusConsumerApp : Started CloudBusConsumerApp in 1.16 seconds (process running for 1.37s)
Received remote event in client-service: HelloWorld

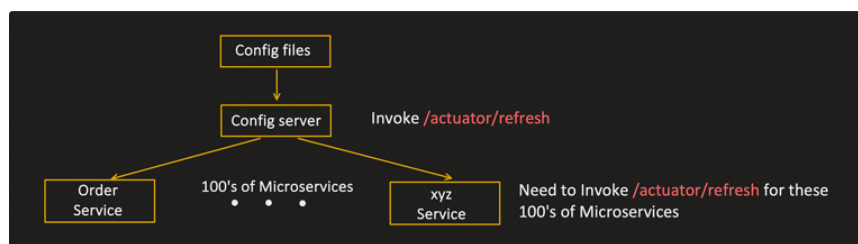
```

Real World Use case of Spring Cloud Bus: Dynamic Configuration Refresh

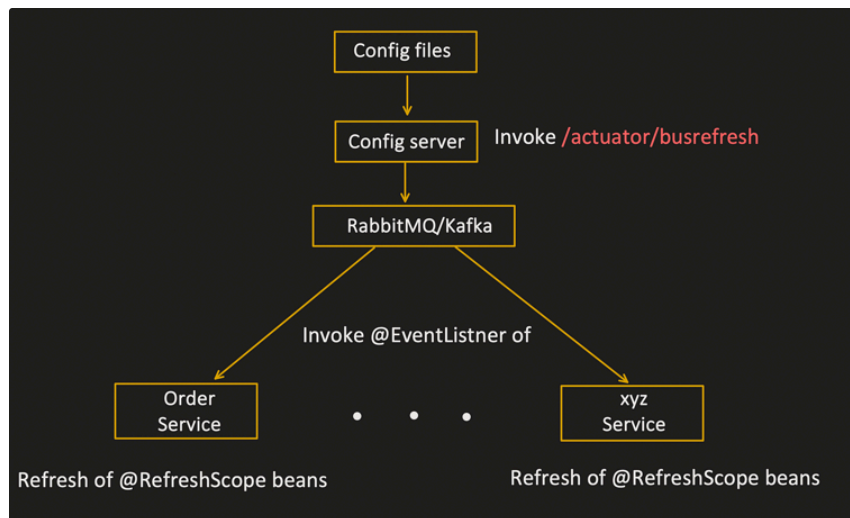
Pre-requisite topic:

1. Central Configuration
2. RefreshScope

Without Spring Cloud Bus:



With Spring Cloud Bus:



Config server:

```

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-config-server</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-bus-amqp</artifactId>
</dependency>
  
```

Only this is added newly (pointing to the last dependency)

```

server.port=8888
spring.application.name=config-server

spring.cloud.config.server.git.uri=https://gitlab.com/shrayansh8/centralconfigs
spring.cloud.config.server.git.username=${GIT_USERNAME}
spring.cloud.config.server.git.password=${GIT_ACCESS_TOKEN}
spring.cloud.config.server.git.search-paths=global, orderservice
spring.cloud.config.server.git.clone-on-start=true
spring.cloud.config.server.git.default-label=main

management.endpoints.web.exposure.include=refresh,health,busrefresh

#message broker and busrefresh endpoint exposure is for dynamic refresh using Spring cloud bus

spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest
  
```

Exposed the actuator endpoint (pointing to busrefresh)

Message broker details (pointing to the rabbitmq properties)

Order Microservices:

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-config</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-bus-amqp</artifactId>
</dependency>
```

Only this is added newly

```
1  spring.application.name=order-service
2  spring.config.import=optional:configserver:http://localhost:8888
3  spring.profiles.active=dev
4
5  #config for all profile (dev, qa, prod etc)
6  custom.message=Hello from local default!
7
8  management.endpoints.web.exposure.include=health,refresh
9
10 #for spring cloud bus
11 spring.rabbitmq.host=localhost
12 spring.rabbitmq.port=5672
13 spring.rabbitmq.username=guest
14 spring.rabbitmq.password=guest
```

Message broker details

No change in controller (already explained in pre-requisite video)

```
@RestController
@RequestMapping("/orders")
public class OrderController {

    private int counter = 0;

    @Autowired
    OrderProperties orderProperties;

    @GetMapping("/fetch")
    public String getOrders() {
        counter++;
        return "fetched orders and message: " + orderProperties.getMessage() +
            " and counter value is: " + counter;
    }
}
```

No change in Configuration Properties class, which need to be updated (already explained in pre-requisite video):

```

@Component
@ConfigurationProperties(prefix = "custom")
@RefreshScope
public class OrderProperties {

    private String message;

    public String getMessage() { return message; }

    public void setMessage(String message) { this.message = message; }
}

```

Start the server for both (Config server and Order Microservice):

```

[config-server] [main] o.s.a.r.c.CachingConnectionFactory : Attempting to connect to: [localhost:5672]
[config-server] [main] o.s.a.r.c.CachingConnectionFactory : Created new connection: rabbitConnectionFactory#4c519e47:0/SimpleConnection@b666366
[config-server] [main] o.s.c.stream.binder.BinderErrorChannel : Channel 'rabbit-1678161997.springCloudBusInput.errors' has 1 subscriber(s).
[config-server] [main] o.s.c.stream.binder.BinderErrorChannel : Channel 'rabbit-1678161997.springCloudBusInput.errors' has 2 subscriber(s).
[config-server] [main] o.s.i.a.i.AmqpInboundChannelAdapter : started bean 'inbound.springCloudBus.anonymous.BjbAmIXoTWSLZd@txo33Wg'
[config-server] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8888 (http) with context path '/'
[config-server] [main] com.concepts.ConfigServerApplication : Started ConfigServerApplication in 4.886 seconds (process running for 5.112)

[order-service] [main] o.s.a.r.c.CachingConnectionFactory : Created new connection: rabbitConnectionFactory#5cdd4548:0/SimpleConnection@16cbb
[order-service] [main] o.s.c.stream.binder.BinderErrorChannel : Channel 'rabbit-440183260.springCloudBusInput.errors' has 1 subscriber(s).
[order-service] [main] o.s.c.stream.binder.BinderErrorChannel : Channel 'rabbit-440183260.springCloudBusInput.errors' has 2 subscriber(s).
[order-service] [main] o.s.i.a.i.AmqpInboundChannelAdapter : started bean 'inbound.springCloudBus.anonymous.0zVsZcmzRfKfBFVpYcKew'
[order-service] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8880 (http) with context path '/'
[order-service] [main] com.concepts.OrderserviceApplication : Started OrderserviceApplication in 6.888 seconds (process running for 7.115)

```

Lets test:

🔍 refreshscope_test
centralconfigs / orderservice / order-service-dev.properties

⚙️
order-service-dev.properties

testing
 Shrayansh Jain authored 19 seconds ago

⚙️
order-service-dev.properties 38 B

1	custom.message=Hello from order dev!
2	
3	

Invoke the endpoint of Order Microservice:

GET
localhost:8080/orders/fetch

Params Authorization Headers (7) Body Scripts Settings

Query Params

	Key	Value	Description
	Key	Value	Description

Body Cookies (1) Headers (5) Test Results 🔄

Raw Preview Visualize

```

1  fetched orders and message: Hello from order dev! and counter value is: 1
    
```

Properly getting the property value from CentralConfigs

Now Update the property in CentralConfigs :

refreshscope_test ▾ centralconfigs / orderservice / order-service-dev.properties

order-service-dev.properties

testing
Shrayansh Jain authored just now

order-service-dev.properties 75 B

```
1 custom.message=Hello from order dev-this is updated to test RefreshScope!
2
3
```

Now, first try to hit the Order API again:

GET ▾ localhost:8080/orders/fetch

Params Authorization Headers (7) Body Scripts Settings

Query Params

	Key	Value	Description
	Key	Value	Description

Body Cookies (1) Headers (5) Test Results | ↺

Raw ▾ ▶ Preview 🖨 Visualize ▾

```
1 fetched orders and message: Hello from order dev! and counter value is: 2
```

Concept & Coding

Now, first invoke “/actuator/busrefresh” endpoint on ConfigServer

POST ▾ localhost:8888/actuator/busrefresh

Params Authorization Headers (7) Body Scripts Settings

Query Params

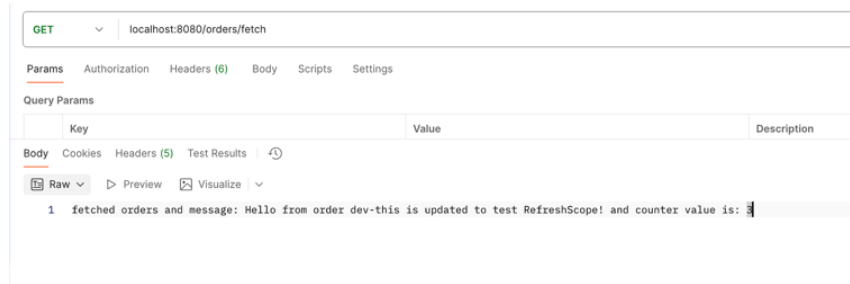
	Key	Value
	Key	Value

Body Cookies Headers (3) Test Results | ↺

Raw ▾ ▶ Preview 🖨 Visualize ▾

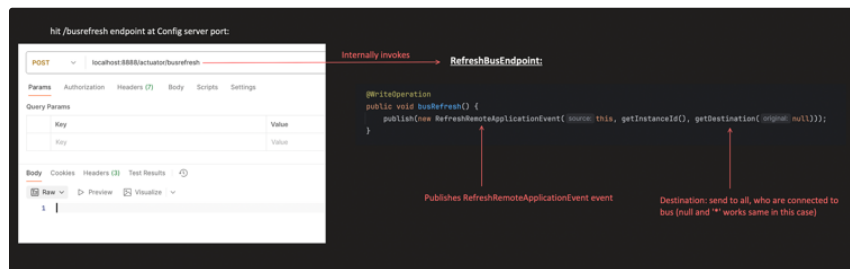
```
1 |
```

Now, first hit the Order API again:



Values refreshed without doing anything at Order Microservice

This is how it works internally:



Notices one thing that, Config Server at this point, just publishes an event, it itself not refreshed the configs yet.

